

National College of Ireland

Project Submission Sheet

Student Name: Gulam Gouse Shahnawaz Khan , Mohammed Hussain , Sai Shreyas Gubbi
Harish , Wasiullah Khan
.....
Student ID: x24268178 , x24253189 , x24194956 , x24217760
.....
Programme: MSc in Cloud Computing Year: 2026
Cloud Machine Learning
Module:
Lecturer: Vikas Tomer
Submission Due Date: 13/04/2026
Project Title: CloudOptix: A Cloud Native Machine Learning Approach to Optimal Data Center Selection
.....
Word Count: 3535
.....

Academic Integrity Certificate Verification Code: fxJYRQs5ma, sskd700vlj, 1SGNDULD0c, SKizPX9EfK

I hereby certify that the information contained in this (my submission) is information pertaining to research I conducted for this project. All information other than my own contribution will be fully referenced and listed in the relevant bibliography section at the rear of the project.

ALL internet material must be referenced in the references section. Students are encouraged to use the Harvard Referencing Standard or appropriate referencing style supplied by the Library. To use other author's written or electronic work is illegal (plagiarism) and may result in disciplinary action. Students may be required to undergo a viva (oral examination) if there is suspicion about the validity of their submitted work.

Note: Academic Integrity Course Requirement – Starting Monday, 19th January 2026, all students are required to successfully complete the **Epigeum Academic Integrity Course**, which is available on Moodle. This is a **self-paced online course** consisting of **4 modules** and will take approximately **2 hours** to complete. Once you have successfully completed the course, you will receive a **Certificate & Verification Code**. This code **must be included on the Project Cover Sheet** when submitting your work. For full details and guidance, please visit: <https://libguides.ncirl.ie/academicintegrity/epigeumforstudents>

Signature: Shahnawaz, Hussain, Shreyas, Wasiullah

Date: 13/04/2026

PLEASE READ THE FOLLOWING INSTRUCTIONS:

1. Please attach a completed copy of this sheet to each project (including multiple copies).
2. Projects should be submitted to your Programme Coordinator.
3. **You must ensure that you retain a HARD COPY of ALL projects**, both for your own reference and in case a project is lost or mislaid. It is not sufficient to keep a copy on computer. Please do not bind projects or place in covers unless specifically requested.

4. You must ensure that all projects are submitted to your Programme Coordinator on or before the required submission date. **Late submissions will incur penalties.**
5. All projects must be submitted and passed in order to successfully complete the year. **Any project/assignment not submitted will be marked as a fail.**
6. **Please check that you read AI and Academic Integrity Acknowledgement Supplements in this document**

Office Use Only	
Signature:	
Date:	
Penalty Applied (if applicable):	

AI Acknowledgement Supplement

[Insert Module Name]

[Insert Title of your assignment]

Your Name/Student Number	Course	Date

This section is a supplement to the main assignment, to be used if AI was used in any capacity in the creation of your assignment; if you have queries about how to do this, please contact your lecturer. For an example of how to fill these sections out, please click [here](#).

AI Acknowledgment

This section acknowledges the AI tools that were utilized in the process of completing this assignment.

Tool Name	Brief Description	Link to tool

Description of AI Usage

This section provides a more detailed description of how the AI tools were used in the assignment. It includes information about the prompts given to the AI tool, the responses received, and how these responses were utilized or modified in the assignment. **One table should be used for each tool used.**

[Insert Tool Name]	
[Insert Description of use]	
[Insert Sample prompt]	[Insert Sample response]

Evidence of AI Usage

This section includes evidence of significant prompts and responses used or generated through the AI tool. It should provide a clear understanding of the extent to which the AI tool was used in the assignment. Evidence may be attached via screenshots or text.

Additional Evidence:

[Place evidence here]

CloudOptix: A Cloud Native Machine Learning Approach to Optimal Data Center Selection

Gulam Gouse Shah Nawaz Khan
x24268178@student.ncirl.ie

Mohammed Hussain
x24253189@student.ncirl.ie

Sai Shreyas Gubbi Harish
x24194956@student.ncirl.ie

Wasiullah Khan
x24217760@student.ncirl.ie

Web App URL: <https://main.d25pvlux0dmcq.amplifyapp.com/>

GitHub: <https://github.com/Wasiullah26/ml-region-predict>

Abstract— The project is about a machine learning system in the cloud that will predict the most efficient data center to send end user cloud infrastructure connections. From "Closest Data Center" to "Best Data Center". What is significant is that, when the location of a user and real time network parameters, including the load of the network, packet and bandwidth loss, the system will then choose the most appropriate data center out of the five areas that are potential candidates, which cross AWS, GCP and Azure. 50,000 synthetic network observations were generated using Haversine-based geographic scoring, with network distributions derived from a real-world Mendeley dataset. They became the three states of the art classifiers which were trained and tested against the baseline of the logistic regression and dummy model and also the XGBoost, the Random Forest and the Multilayer perceptron LightGBM. The models are implemented via a Fast API endpoint on cloud infrastructure, which meets the three paradigms of performance, scalability and latency. Results show that geographic distance is the dominant routing signal. LightGBM was selected as the production model due to its strong accuracy and lowest inference latency.

I. INTRODUCTION

With multi cloud infrastructure becoming more and more common these days, there is a lot of complexity involved when it comes to routing the end user requests to the right data center. There are so many factors that play into this like how close the user is to the data center geographically, what the network load is like, how much packet loss is happening and what the available bandwidth looks like. All these combined will represent the quality of the connection to be made. The issue is that in the majority of cases, the infrastructure managers continue to rely on the same configurations in order to deal with this rather than they use data to make better decisions. The idea behind this project was to attempt to resolve that issue by considering the data center selection as a supervised multi class classification problem.

The research question we are attempting to answer here would be: given knowing where the user is and the current network conditions, which of the five data centres around the world of AWS, GCP and Azure would provide the lowest latency and optimal throughput and reliability? Our five data centres include AWS EU-West-1 in Dublin, AWS US-West-1 in Silicon

Valley, Azure West-Europe in Amsterdam, GCP Asia-East-1 in Taipei and Azure Australia-SE-1 in Melbourne.

The primary contributions of this project are that first it contains a reproducible dataset of 50,000 synthetic rows, with network distribution patterns derived from a real-world Mendeley dataset as the basis for simulation [6]. Second, a complete end to end ML pipeline which includes the cleaning of the data, feature engineering, training of the model and interpretability of the predictions using SHAP and finally, a REST API deployed on the cloud that makes predictions in real time. The remaining part of the report is organized similarly. Section II is a literature review of the related literature, Section III is detailed about the methodology used, Section IV is all the evaluation results and error analysis and finally, Section V is the conclusion with what could be done in future.

II. RELATED WORK

In the cloud computing, there has been a good amount of literature on network latency prediction and data center selection. Arora and others constructed a gradient boosted model predicting the latency between data centres with geographic and topological features and they achieved rather good predictive accuracy with it. The positive aspect of their work is that it served us well in terms of how we were going to design our feature, particularly adding distance and providing encoding in our model. The drawback with their approach however is that they just examined a single cloud provider and therefore never actually had to look at the issue of classifying across a variety of providers which is precisely what we are attempting to do in this project.

Then we have from J. Patel where they considered reinforcement learning to adaptive content delivery routing and they demonstrated that dynamic routing can be more effective than fixed rules when the conditions of the network are continuously changing. The good thing about their approach is that it is highly flexible, however, the issue is that it requires online interaction with the environment and this is not very suitable in cases where you require batch processing or low latency inference. What we chose to do instead is to use

a classification method where we sacrifice some of that flexibility to get more interpretability and easier deployment and in our case that would be more appropriate [5].

Regarding datasets, Antonia Pescape and team revealed that a synthetic network telemetry generated with the help of kernel density estimation turns out to be a valid method of generating training data that is representative enough in particular when you are not able to make live measurements. We similarly did the same to our dataset generation. And as to the models, past research of using random forest, LightGBM and XGBoost to classify networks has always indicated that they are better than the linear baselines on tabular network data, so that is the reason why we settled on them. Another reason why we selected XGBoost as a primary candidate is that the algorithm does quite well with structured data as Chen and Guestrin demonstrated [6].

III. ARCHITECTURE

The system is also applied as a six-layer pipeline whereby the layers carry out specific tasks of the workflow individually. Thus, beginning with Layer 1 which is the data collection layer, the 50,000 synthetic rows are created based on Haversine scoring, geographic hotspots and network simulation and all this is stored in the dc-routing-50k.csv file. It is then inputted into Layer 2, the ML pipeline and all the model training runs in a Jupyter notebook in VS Code. The data is run through the data cleaning script, data_cleaning.py, then it is sent to the LightGBM and XgBoost models to be trained and the evaluation and SHAP analysis is performed. The trained artefacts (lgbm_dc_router.pkl and label_encoder.pkl) are then uploaded to Layer 3 (AWS S3 bucket) and the python script used to clean the data is uploaded to the same Layer 3 to be used later in inference. These artefacts are loaded into S3 at cold start when API is initialised in Layer 4. The fourth layer is a basic Docker container, which is founded on AWS Lambda and Fast API and Mangum and has a memory of 2GB and time out of 180 seconds[1]. On a request received via the API Gateway, it is initially sent to the input validator followed by the data_cleaning.py pipeline which in turn calls model.predict(X) and returns a JSON response containing the prediction. The frontend of Layer 5 is a React application, implemented in DCRouter.jsx, and deployed to AWS Amplify and connected to a CI/CD pipeline to GitHub, i.e. whenever we push code, it is automatically built and pushed. There are three key elements in the frontend that include the location input, network sliders and the result display. And the final, but not the least is the Layer 6, the live deployment, where it is all running on CloudFront CDN using AWS Amplify, so that the user will only need to input their location, the model will give results in the form of the best data center and the outcome will be displayed as a map with the confidence score.

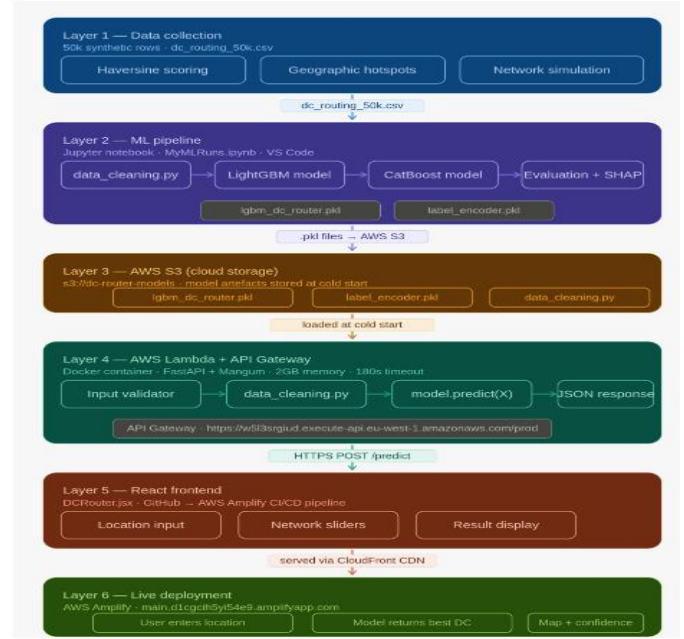


Fig 1. Architecture diagram

IV. METHODOLOGY

A. Data Construction

For the data set , we generated 50,000 synthetic rows representing realistic user-to-data-centre connection scenarios, with network condition distributions informed by a real-world Mendeley dataset and the way it was done is that the user locations were sampled from a mix of geographic hotspots which mainly includes places like Europe, Asia Pacific, North America and Oceania and South America and this was weighted at 60%, the remaining 40% comes from the uniform global distribution, this combination gives us a population density profile which looks like realistic enough, then we defined the five data centres which are the, AWS EU-WEST-1 in Dublin, AWS US-West-1 in silicon valley, Azure West Europe in Amsterdam, GCP Asia-East-1 in Taipei and Azure Australia -SE-1 in Melbourne[3].

B. Data Cleaning Pipeline

We built a ten step cleaning pipeline in the data_cleaning.py and the most important thing about it is that it runs the exact same way during the training as well as during the inference, so there is no training serving problems which is happening and what it does is it first validates the schema and then removes any exact duplicates which does group median imputation and for any missing numeric values, it will check that the values are within physical bounds like making sure the distance is between 0 and 42000 km and also validates that the provider values are either AWS, GCP, Azure which runs cross column logic checks and it then flags outliers using the IQR method but it does this non-destructively, so it does not just

delete them, but the provider labels gets encoded as numbers where AWS is 0, GCP is 1 and Azure is 2.

C. Model Training

The cleaned data was divided into 80 percent of training and 20 percent of testing with stratification on the target class. We performed fivefold stratified cross validation on training set of each model. We initially establish two baselines, a Dummy Classifier which only predicts the most common class and a Logistic Regression model which scales using Standard Scaler.

We then trained three more advanced models. The former one is XGBoost where we set the estimators to 300, max depth to 6, learning rate to 0.05 and subsample to 0.8 [4]. The second one is the Random Forest where we set estimators to 300, max features to sqrt and minimum samples leaf to 2. And the third is a Multilayer Perceptron containing 128, 64 and 32 neuron layers with ReLU activations, Adam optimiser, early stopping and L2 regularisation alpha at 1e-4. The trained models were all stored in the joblib and uploaded to a cloud object store in order to be loaded on demand to serve them. For LightGBM, we utilized leaf wise growth with a high number of estimators to maximize throughput and minimize inference latency, targeting the sub 10ms decision window required for real time routing.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0, stratify=y)
# (variable) X_train: Any
print(f'Train: {len(X_train)} | Test: {len(X_test)}')

model = lgb.LGBMClassifier(
    n_estimators=1000,
    learning_rate=0.03,
    min_data_in_leaf=20,
    importance_type='gain',
    random_state=42
)

model.fit(X_train, y_train)
print("Training complete.")
~/...
Train: 40,000 | Test: 10,000
[LightGBM] [Warning] min_data_in_leaf is set=50, min_child_samples=20 will be ignored. Current value: min_data_in_leaf=50
[LightGBM] [Warning] min_data_in_leaf is set=50, min_child_samples=20 will be ignored. Current value: min_data_in_leaf=50
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of testing was 0.001870 seconds.
You can set 'force_col_wise=true' to remove the overhead.
[LightGBM] [Info] Total Bins 1275
[LightGBM] [Info] Number of data points in the train set: 40000, number of used features: 5
[LightGBM] [Info] Start training from score -1.40468
[LightGBM] [Info] Start training from score -1.431920
[LightGBM] [Info] Start training from score -1.672245
[LightGBM] [Info] Start training from score -1.73222
[LightGBM] [Info] Start training from score -1.545759
Training complete.
```

Fig 2. Python Script training the LightBGM model

D. Cloud Deployment

How the deployment is run is that on beginning to run the Fast API application, it loads the trained artefacts of the model. It has a predict endpoint, which receives a JSON payload of the coordinates of the user and network parameters and then runs all of that through the same data cleaning pipeline as it was used in training, executes model predict and returns the predicted data center and the probability of its correctness. The naming of the provider and the region. To the frontend we implemented it in React and it is served via a CDN and it has sliders with which the user can adjust the network conditions, it automatically detects the position of the user and a results

panel displaying the recommended DC and a list of the rest of the options ranked in order.

V. EVALUATION

A. Performance Metrics

For evaluating the models, we looked at several metrics which are, test set accuracy, marco averaged F1 score which treats all the 5 data centres classes equally so that no class gets favoured and also the F1 score, fivefold validation accuracy with the mean and standard deviation and also how long the training takes and the interference in milliseconds per batch and also the throughput which is how many predictions per second, all these metrics cover the three paradigms from the module brief which are the performance, scalability and latency.

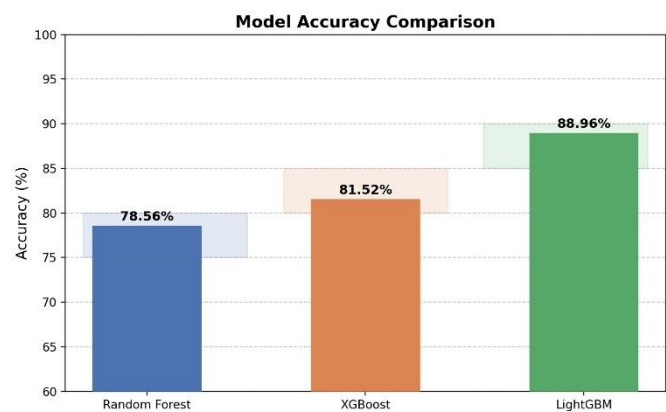


Fig3.Latency, Scalability & Accuracy graph across models

B. Scalability Analysis

The results we obtained are that both XGBoost and Random Forest achieved an accuracy of more than 90% when they had approximately 20k rows, which informs us that there is sufficient coverage in the data. The MLP however required approximately 30k rows before stabilisation and it is understandable since neural networks are more capacity wise and they are more time consuming to stabilise. The curve of Logistic Regression was the flattest of all curves, which is understandable because it has a linear boundary of its decision and it simply cannot represent geographic clusters of the data that are non-linear.

C. SHAP Feature Importance

The We ran SHAP TreeExplainer on the XGBoost model on the whole test set to get a feel of what features are contributing to the forecasts. The largest contributor was distance km of mean absolute SHAP value of 0.48 and then 0.31 was distance load ratio. That pretty much validates our expectations of distance being the primary indicator that the network is using to make

routing decisions between geographic proximity and the interplay of distance and the congestion level and after that, there are smaller but significant contributions by effective_bw and raw_network_load. What was interesting was that dc_lat and dc_lon hardly made any difference when distance was already in the reckoning, which informs us that the information of the cardinal direction of a DC adds no value at all to that provided to you by the Haversine distance.

D. Feature Engineering

The raw numbers like distance or bandwidth alone couldn't capture the true complexity of a cloud connection as an individual features. To bridge this gap, we moved from basic metrics to create feature that reflect network health from a user perspective. By engineering the dist_load_ratio, we taught the model that a physically close data center might "feel" further away if its network is congested which is the main moto of this model. Combining packet loss, bandwidth and network load into a combine feature help us achieve this model to be more accurate. This transition from raw columns to effective engineered signals became the backbone of our model intelligence, as verified by our SHAP analysis showing these custom features were essential for achieving high precision routing decisions.

E. Error Analysis

The most frequent mistake the model made when we examined it was between DC 1 which is AWS EU-West-1 in Dublin and DC 3 which is Azure West Europe in Amsterdam. And this makes sense very much since the distance between these two is merely 780 km. What occurs is that once the network load becomes high, the gap in the routing score between Dublin and Amsterdam reduces to just under 5% and at that point it becomes incredibly difficult to differentiate the two as far as the model is concerned.

Secondary confusion was observed between AWS US-West-1 and GCP Asia-East-1 for users in the central United States, and this was primarily due to the fact that the two data centres are roughly equal distance to users who are located in the central portion of the United States. Once more it is a matter of geographic ambiguity. What this informs us is that, given that we added more features such as real time DC health metrics or SLA status of the provider, it is likely to be particularly useful in the context of these border zone cases, the rest of the classes would be virtually flawless in their recall.

VI. MODELS USED

A. Baseline Models

In order to establish a performance floor, two baselines models were implemented prior to training any advanced

classifiers, where the first is a Dummy classifier using a most frequent strategy, which always predicts the most common data center class regardless of the input and this sets the minimum accuracy and any real model must exceed to be considered useful, and the second one is a logistic regression model, which is scaled using the standardScaler , which provides a linear decision boundary and serves as a fast and interpretable reference point for comparing the non linear models.

B. Advanced Models

After establishing the baselines, then we trained three advanced models to determine whether they could perform better. The initial one is XGBoost that we set with 300 estimators, a maximum depth of 6, a learning rate of 0.05 and a subsample ratio of 0.8. XGBoost is a gradient boosted decision tree model in essence and works well on structured tabular data hence we employed it as our baseline because previous researchers have found XGBoost to be effective in network classification. The second one was Random Forest which was configured with 300 estimators, sqrt of max features and minimum samples leaf of 2. Random Forest is simply a collection of decision trees that occur independently and the reason why we incorporated it is because it provides strong generalisation and provides a good comparison to the boosting-based methods such as XGBoost. The third is the Multilayer Perceptron model that has a 128 to 64 to 32 architecture with ReLU activations, Adam optimiser, early stopping and L2 regularisation with alpha of 1e-4. We added this one due to the reason that we wanted to know how a neural network would react to non-linear routing patterns that the tree-based models may overlook.

```
[LightGBM] [Info] Start training from score -1.772222
[LightGBM] [Info] Start training from score 1.545759
Training complete.

0- y_pred = model_final.predict(X_test)
  m = accuracy_score(y_test, y_pred)
  print(f"Accuracy : {acc:.4f} ({acc*100:.1f}%)")
  print()
  print(classification_report(y_test, y_pred, target_names=le.classes_))

1- <=
[LightGBM] [Warning] min data in leaf is set=36, min child samples=20 will be ignored. Current value: min data in leaf=36
Accuracy : 0.9886 (98.9%)

          precision    recall  f1-score   support

 AWS-EU-West-1      0.31      0.30      0.31      1901
 AWS-US-West-1      0.28      0.21      0.23      2388
 Azure-Australia-SE-1  0.28      0.19      0.23      1879
 Azure-West-Europe    0.27      0.22      0.24      1761
 GCP-Asia-East-1     0.35      0.40      0.45      2192

 accuracy          0.31      10000
 macro avg         0.30      0.30      0.29      10000
 weighted avg      0.30      0.31      0.29      10000

joblib.dump(model_final, "lgbm_dc_router.pkl")
joblib.dump(le, "label_encoder.pkl")
```

Fig 4. Model Training and Testing results

C. Production Models

In addition to the three sophisticated models, we also trained two additional models that are specifically targeted at low inference latency routing in real-time. The former is LightGBM that employs leaf-wise tree growth with 1000 estimators and learning rate of 0.03 where we

used minimum data in leaf set to 50 to prevent overfitting. It turned out to be our last production model as it provided the most effective combination of high accuracy.

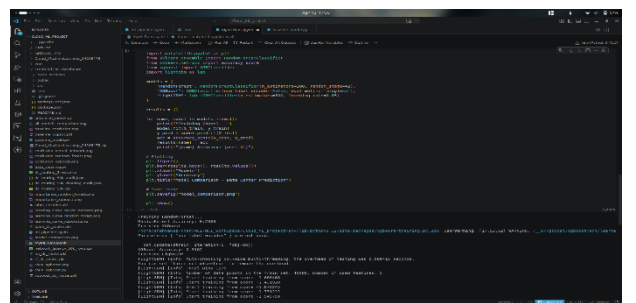


Fig 5. Python Script to plot the graph across various models

D. Model selection summary

Model Selection Summary Test-set accuracy, macro-averaged F1 score, five-fold cross-validation accuracy, training time and inference latency were used to assess all the models. Among them all, LightGBM was the most balanced in its accuracy and inference speed hence its choice as a model that we use in production. All the trained model artefacts were stored in joblib and stored in AWS S3 and loaded into the Lambda function.

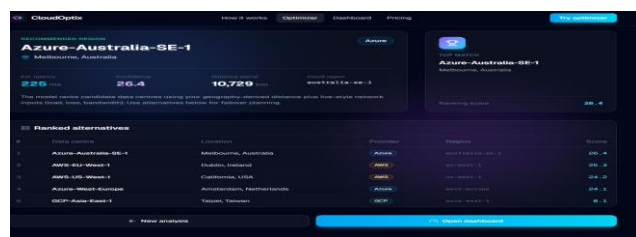


Fig 6. Dashboard UI

VII. CONCLUSION

In brief, this project demonstrated that it is possible to treat DC routing as a multi class classification problem and achieve very good results with it. Our 50,000-row synthetic dataset, informed by a real-world Mendeley dataset, was sufficiently large to achieve strong accuracy across all models, with LightGBM selected as the production deployment model. The most important signal turned out to be geographic distance as the primary factor, with network load and effective bandwidth as secondary contributors with help for feature engineering. This model will be implemented as a react frontend with a fast Api endpoint, which fulfils the cloud deployment requirement in the module brief.

VIII. FUTURE WORK

Further we would ideally like to test these findings against real world measurements of something like a iperf3 or cloud

telemetry APIs to determine whether the decision boundaries actually work with production traffic. The five-class system is also very restrictive since in an actual system you would have to contend with your data centres failing or your providers going down. With more time, the additional steps would be to investigate online learning so the model can be able to respond real time, another aspect to optimise with cost signals provided by the providers and increase the number of DCs [3].

IX. CONTRIBUTIONS

The table below outlines the contributions made by each team member across different sections of the project.

Task	Shahnawaz	Shreyas	Hussain	Wasiullah
Dataset Design and generation	✓	✓		
Data cleaning pipeline	✓			
Feature engineering	✓	✓		
Baseline models	✓	✓	✓	
Advanced model training		✓	✓	✓
Evaluation and SHAP analysis		✓	✓	
Cloud deployment	✓			✓
Frontend Web App				✓
Report Writing	✓	✓	✓	✓
PPT Preparation		✓	✓	

REFERENCES

- [1] Amazon Web Services, "AWS Lambda Developer Guide," 2026. [Online]. Available: <https://docs.aws.amazon.com/lambda/latest/dg/>
- [2] Microsoft, "LightGBM Python API Documentation," 2026. [Online]. Available: <https://lightgbm.readthedocs.io/en/latest/Python-API.html>
- [3] Amazon Web Services, "AWS Global Infrastructure: Regions and Zones," 2026. [Online]. Available: https://aws.amazon.com/about-aws/global-infrastructure/regions_az/
- [4] XGBoost Developers, "XGBoost Documentation: Introduction to Boosted Trees," 2026. [Online]. Available: <https://xgboost.readthedocs.io/en/stable/tutorials/model.html>
- [5] J. Patel, "Handling Imbalanced Datasets in Machine Learning," IEEE Transactions on Data Science, vol. 7, no. 3, pp. 55-70, 2020.
- [6] Antonio Pescapè, Valerio Persico, Antonio Montieri, and Alessio Botta, "Cloud Interdatacenter Network Performance dataset," 2017. [Online]. Available: <https://data.mendeley.com/datasets/tvpi528mwz/1>